DAY 4 · MODULE 7 · 25 MIN

Day 4 Lab Kickoff

Three parts, one scenario — workflow basics, then orchestrations, then evaluation

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/ · +1 in notes

What you'll build

- The same three roles — Planner, Retriever, Critic:
  - Part A — a raw WorkflowBuilder graph, by hand, no loop
  - Part B — the SAME roles, three ways: SequentialBuilder's shortcut, a custom graph with a revision loop, GroupChatBuilder's alternative fix
  - Part C — the golden set against all three of Part B's constructions
- **Optional stretch** — a Ticket agent that files a real Azure DevOps work item (Day 3's MCP path) when the Critic flags a documentation gap
- Part C runs the same golden set against every construction from Part B — same questions, one comparison table

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/

Part A — Workflow basics

- Wrap Planner, Retriever, Critic in AgentExecutor; wire them straight-line with WorkflowBuilder + add_edge — no loop, no conditional edges yet
- output_from=[critic] designates the Critic's response as the ONE workflow output — without it, all three executors' responses would count
- Run it once on a single question; visualize the graph with WorkflowViz(workflow).to_mermaid() (Module 3's own visualization tooling)
- No golden set here — that's Part C's job. Part A is purely "how is a workflow actually built"

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/executors

Part B — Orchestrations

```
# Construction #1 — the shortcut for Part A's graph
SequentialBuilder(participants=[planner, retriever, critic]).build()

# Construction #2 — fixes its limitation
builder = WorkflowBuilder(start_executor=planner)
builder.add_edge(planner, retriever)
builder.add_edge(retriever, critic)
builder.add_edge(critic, planner, condition=lambda result: not result.approved)
```

Same limitation as Part A: Sequential's Critic runs once, with nowhere to send a rejected verdict. Construction #2 fixes it with a conditional loop back to the Planner.

Source: https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/sequential · +1 in notes

Part B — Group Chat, a second fix

1 Rebuild

same three roles as plain participants, now with GroupChatBuilder(participants=[...], orchestrator_agent=..., termination_condition=...)

2 Orchestrator decides, not Python

an LLM agent reads the shared conversation and picks who speaks next; no condition function to write

3 Context Synchronization does the work revision_gate had to do manually

the Planner already sees the Critic's rejection when re-selected, because the conversation is shared

4 The guardrail lives in termination_condition

GroupChatBuilder gives you exactly one hook to control termination, not a graph of edges to route through

Source: https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/group-chat

Part B's guardrail is required for both fixes

- Neither a conditional edge nor an LLM orchestrator has a built-in max_iterations — unlike AgentLoopMiddleware's single-agent loop, nothing stops either loop automatically
- Construction #2: a counter in workflow state (ctx.set_state/get_state), checked by a small executor since condition functions never get ctx
- Construction #3: termination_condition itself is the guardrail — a message-count cap alongside the "Critic approved" check, in one function
- **Prove both fail safely** — a bounded, graceful stop, not an unbounded token burn. This is Module 6's advice, applied to the two loops this lab can actually trigger

Source: https://learn.microsoft.com/en-us/agent-framework/agents/looping

Part C — Evaluate and compare

- Imports Part B's three build_workflow_*() functions directly — no new orchestration code
- Runs the SAME golden-set slice against all three constructions: Sequential, custom graph, Group Chat
- Reports a comparison table: approved rate for each, plus guardrail-trip rate for construction #2 specifically (construction #3's guardrail has no distinct signal from a plain rejection)
- **Reflect** — which approach fit best, and why — write it down, it's part of Done

Source: https://learn.microsoft.com/en-us/agent-framework/agents/evaluation

Build one golden set, use it for Part C

- ~15 realistic questions, each with expected citations/answers as ground truth
- Cover the real branches: a question that grounds cleanly on the first pass, one that needs the Critic's revision loop, and a general-knowledge question needing no retrieval at all
- Build it once, before Part C — every construction runs against the exact same set, so the comparison means something

Source: https://learn.microsoft.com/en-us/agent-framework/agents/evaluation

Run the trajectory evaluation

- **Plan quality** — did the Planner's decomposition make sense for the question?
- **Retrieval recall** — did the Retriever actually find the right grounding?
- **Critic accuracy** — both decisions: the reject-and-loop call (constructions #2/#3 only — construction #1 never loops), *and* the final structured Answer
- **End-to-end task success** — did the whole workflow produce a usable answer? (this lab's actual implemented metric: approved / not approved / guardrail tripped)
- **Cost per successful outcome** — tokens summed across every agent, divided by successful cases (Module 5 names this; not implemented in this lab's Part C — a real extension, flagged rather than assumed)

Source: https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators

Prerequisites

Day 3 lab complete

a working single agent with memory, streaming, structured outputs, and MCP

Bundled reference docs

included in the repo (labs/day4/python/data/docs/); nothing to provision, no live knowledge base required

A Foundry judge model deployment

for cost/trajectory evaluation

(Optional stretch only)

Day 3's Azure DevOps MCP path, for the Ticket agent

Source: https://learn.microsoft.com/en-us/agent-framework/agents/evaluation

Definition of done

Area	Done when
Part A	The graph runs end-to-end and prints the Critic's verdict; the Mermaid diagram matches the graph
Part B	All three constructions run on the same hard question; constructions #2/#3 both recover from construction #1's limitation
Part B guardrail	Triggers cleanly in a stress test for BOTH constructions #2 and #3 — required, not stretch
Part C	Golden set runs against all three constructions; a comparison table reports approved rate side by side
Reflection	Which construction fit best, and why — committed to the repo
Stretch	Ticket agent files a real ADO work item when the Critic flags a gap

Source: https://learn.microsoft.com/en-us/agent-framework/agents/evaluation · +1 in notes

Takeaways

- ✓ You build a workflow by hand before you use the prebuilt shortcut — Part A first, Part B second, deliberately in that order.
- ✓ You build the same three roles three different ways — Sequential, a custom graph, Group Chat — and watch the same limitation get fixed two different ways.
- ✓ You measure every construction against the same golden set in one dedicated evaluation pass, so the differences you see are real, not noise.
- ✓ You build the guardrail Module 6 argued for — required for both fixes, because nothing else bounds either loop.

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/ · +1 in notes